

4.1 — Design Prompts with Explicit Criteria to Improve Precision

Concept

Vague prompts produce vague results. When you ask Claude to "review this code" or "check for issues," you are delegating the definition of quality to the model. Claude will form its own interpretation of what matters, which may differ from yours, vary between runs, and drift across models or versions. The result is inconsistent output that is hard to evaluate and harder to act on.

Explicit criteria replace subjective delegation with objective enumeration. Instead of telling Claude what to feel ("find problems"), you specify exactly what to check ("check for SQL injection, XSS, and missing authentication"). This shift has three practical benefits:

1. **Repeatability** — the same criteria produce the same kind of output on the same input.
2. **Evaluability** — you can measure whether Claude found what you defined, rather than judging the quality of an opinion.
3. **False positive management** — a precise scope prevents Claude from flagging stylistic preferences as security vulnerabilities, or performance observations as correctness bugs.

The False Positive Problem

False positives are expensive. In automated pipelines they block deployments, page on-call engineers, or surface noise that erodes trust in the system. In human-facing tools they exhaust reviewers. The root cause is almost always an under-specified prompt.

When a prompt says "find security issues," Claude may flag:

- A long function (readability concern, not a vulnerability)
- A `console.log` statement (data leak risk, but also a debug artifact)
- An unvalidated variable that is, in context, a compile-time constant

An explicit criteria list constrains the search space. Claude still applies judgment within each criterion, but it does not invent new categories of concern. The boundary is yours to define.

Enumerated Criteria Lists

The most reliable pattern is a numbered or bulleted list of named checks, each with a definition. Definitions should answer: what does a positive finding look like? What does it not look like?

This is the mental model: **Specificity beats vagueness. Examples beat instructions. Schemas beat parsing.**

- Specificity: name the thing you want checked, not the class of things.
- Examples: show what a finding looks like before asking for findings.

- Schemas: if you need structured output, provide a JSON schema rather than asking Claude to invent a format.

Code Example

Vague prompt — inconsistent, noisy results

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

async function vagueReview(code: string): Promise<string> {
  const response = await client.messages.create({
    model: "claude-sonnet-4-5",
    max_tokens: 1024,
    messages: [
      {
        role: "user",
        content: `Review this code and find any issues:\n\n${code}`,
      },
    ],
  });

  const block = response.content[0];
  return block.type === "text" ? block.text : "";
}
```

This prompt will return different categories of feedback on different runs. You cannot reliably parse, aggregate, or act on the output in an automated pipeline.

Explicit criteria prompt — precise, structured, actionable

```
import Anthropic from "@anthropic-ai/sdk";

interface SecurityFinding {
  criterion: string;
  line: number | null;
  severity: "critical" | "high" | "medium";
  description: string;
  remediation: string;
}

interface ReviewResult {
  findings: SecurityFinding[];
  criteriaChecked: string[];
}
```

```

}

const SECURITY_CRITERIA = [
  {
    id: "SQL_INJECTION",
    description:
      "User-controlled input concatenated into a SQL string without parameterization.",
    notThis: "Static query strings or ORM methods with bound parameters.",
  },
  {
    id: "XSS",
    description:
      "User-controlled input rendered into HTML without sanitization (innerHTML, dangerouslySetInnerHTML, etc.).",
    notThis: "Text content assignment (textContent, innerText) is safe.",
  },
  {
    id: "AUTH_BYPASS",
    description:
      "An authenticated route or protected resource reachable without a valid session or token.",
    notThis: "Public endpoints intentionally lacking authentication.",
  },
  {
    id: "HARDCODED_SECRET",
    description:
      "An API key, password, or token literal in source code (not in environment variable reference).",
    notThis: "Placeholder strings like 'YOUR_API_KEY_HERE' or empty strings.",
  },
] as const;

async function preciseSecurityReview(code: string): Promise<ReviewResult> {
  const criteriaBlock = SECURITY_CRITERIA.map(
    (c) => `- **${c.id}**: ${c.description} (not this: ${c.notThis})`
  ).join("\n");

  const prompt = `You are a security code reviewer. Check the following code against exactly the following criteria:

${criteriaBlock}

For each criterion, either report a finding or confirm it is clear. Do not flag issues outside the provided code.

Return your response as JSON matching this schema:
{
  "findings": [
    {
      "criterion": "<criterion id>",
      "line": <line number or null>,
      "severity": "<critical|high|medium>",
    }
  ]
}
```

```

        "description": "<what you found>",
        "remediation": "<specific fix>"
    }
],
"criteriaChecked": [<list of criterion ids you checked>"]
}

```

Code to review:

```

\\`\\`
${code}
\\`\\`;

```

```

const response = await client.messages.create({
  model: "claude-sonnet-4-5",
  max_tokens: 2048,
  messages: [{ role: "user", content: prompt }],
});

const block = response.content[0];
if (block.type !== "text") throw new Error("Unexpected response type");

const jsonMatch = block.text.match(/\[([^\s\S]*)\]/);
if (!jsonMatch) throw new Error("No JSON found in response");

return JSON.parse(jsonMatch[0]) as ReviewResult;
}

```

The explicit version gives you a result you can parse, store, diff across commits, and route to the right team. The `criteriaChecked` field confirms Claude did not skip a criterion silently.

Common Mistakes

- 1. Using category names without definitions.** Writing "check for injection vulnerabilities" still delegates interpretation. Define what counts as a positive: which APIs, which patterns, what context makes it exploitable.
- 2. Listing too many criteria.** More than eight to ten criteria in a single prompt increases the chance of overlooked items and dilutes attention. Group related checks into sub-prompts or batch reviews.
- 3. Omitting negative examples.** Telling Claude what a finding looks like is necessary but not sufficient. Equally important: tell it what does not qualify. This is the primary lever for false positive reduction.
- 4. Asking for free-form output then trying to parse it.** If you need structured output, provide a schema. Free-form prose is appropriate for human readers; it is fragile for automated pipelines. Use `response_format` or embed the schema in the prompt and parse the JSON block.
- 5. Conflating severity with category.** Severity is a separate axis from the criterion. Define each criterion first, then specify how severity should be assigned within that criterion (e.g., "SQL injection in a public endpoint is

critical; in an admin-only route it is high").

References

- Anthropic Prompt Engineering Guide — [Clarity and Specificity](#)
- Anthropic Docs — [Structured Output with JSON](#)
- Domain mental model: *Specificity beats vagueness. Examples beat instructions. Schemas beat parsing.*